arXiv:2610.13764v1 [stat.ML] 9 Oct 2026

How Well Do Prior-Data Fitted Networks Amortize Bayesian Inference?

Manuel Hülskamp **Julius Kobialka** **Emanuel Sommer** **Thomas Nagler** **David Rügamer**
LMU Munich and Munich Center for Machine Learning

Abstract

Prior-data fitted networks (PFNs) are trained to approximate the Bayesian posterior predictive distribution (PPD) of a synthetic prior, yet results about the approximation quality are surprisingly thin. Taking an empirical risk minimization perspective, we decompose the deployment error of PFNs into prior, context, and amortization error, and measure the latter against a reference PPD under a realistic prior. Results show that PFNs generally recover the targeted PPD very well. Nevertheless, we find that parts of the residual amortization error persist, especially in *rare* settings under the prior pretraining distribution, even when scaling the model size, which manifests in disagreement between independently pretrained PFNs and leaves uncertainty about the predictor at inference time. As a remedy, we study multiple types of Bayesian model averages and find that these can consistently reduce this uncertainty and improve predictions up to full-size TabICLv2 models.

1 INTRODUCTION

Prior-data fitted networks (PFNs) have emerged as powerful foundation models for tabular data (Müller et al., 2021; Hollmann et al., 2025). By learning to approximate Bayesian posterior predictive distributions (PPDs) via in-context learning (Brown et al., 2020) during pretraining, they perform inference on new downstream datasets in a single forward pass. Given their success, many works have proposed strategies to improve predictive performance, scale to larger datasets, or transfer the approach to other tasks (Qu et al., 2026; Zhang et al., 2025b; den Breejen et al., 2024; Thomas et al., 2024; Xu et al., 2025; Liu and Ye, 2025; Ma et al., 2026; Robertson et al., 2025; Balazadeh Meresht et al., 2025). The Bayesian interpretation is central to this success. A PFN is trained on datasets drawn from a synthetic prior, and its training objective is minimized by the PPD of that prior. In other words, a PFN is trained to minimize its risk under its own prior, and the PPD is the optimal predictor there.

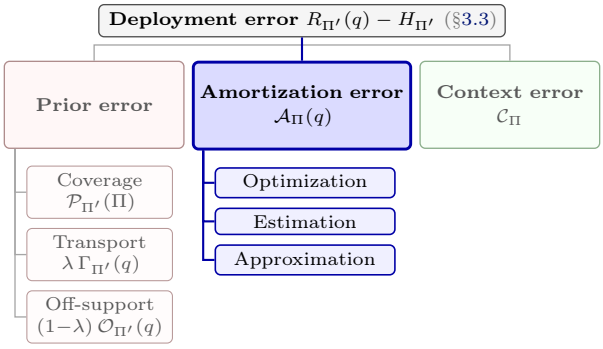


Figure 1: Decomposition of the deployment error of a PFN (Proposition 2). This paper studies the amortization error, the only term controlled by pretraining, which splits into optimization, estimation, and approximation error (Eq. (8)). The context and prior error support this analysis: the context error is incurred even by the exact PPD, and the prior error arises when the deployment tasks differ from the prior (see Section 3.3).

Surprisingly, the evidence that trained PFNs closely track the PPD they amortize is thin and mostly focused on whether *transformers can do Bayesian inference* at all via the PPD (Müller et al., 2021) and not how well they recover it. Moreover, pretraining is typically reduced to first-order optimization of a single network, on the grounds that effectively unlimited synthetic data should remove any uncertainty about the outcome. Whether this holds is a question about amortization uncertainty. Existing work on uncertainty in PFNs (Nagler and Rügamer, 2025; Fortini et al., 2026) studies the uncertainty that remains for a *fixed* network at inference time and is, by construction, blind to it.

Furthermore, when PFNs are deployed on real data, the model likely faces a different distribution of tasks, and the optimal predictor changes with it. Hence, deployment is a distribution shift from the pretraining prior to the real-world task distribution, which further

complicates the error source analysis of deployed PFNs.

Our Contribution In this work, we make the following contributions to study the sources of error and uncertainty in PFNs with a focus on amortization:

1. A decomposition of the error of any PFN under a deployment distribution, splitting into prior, context and amortization error.
2. A study of the amortization error of PFNs under its training prior, including a comparison to a realistic reference PPD (Section 2).
3. An extended investigation of the amortization error of PFNs under a deployment task distribution (Section 3).
4. An analysis of approximate Bayesian model averages over PFN weights (Section 4): Deep ensembles consistently improve predictions, whereas sampling and variational learning add little.

2 AMORTIZATION ERROR

2.1 Setup

Following Nagler (2023), a *task* is a law p over $(\mathbf{X}, Y) \in \mathcal{X} \times \mathcal{Y}$ with covariate law $p_{\mathbf{X}}$ and conditional $p(y \mid \mathbf{x})$, and a dataset $\mathcal{D}_n = \{(\mathbf{X}_i, Y_i)\}_{i=1}^n$ consisting of n independent and identically distributed (iid) draws from it. A PFN assumes a *prior* Π over pairs (p, N) of a task and a context size, whose marginal over N is the *size distribution* Ψ. Given $(p, N{=}n)$, the context is $\mathcal{D}_n \sim p^{\otimes n}$, and the PFN targets the posterior predictive distribution (PPD)

$$\pi_\Pi(y \mid \mathbf{x}, \mathcal{D}_n) = \int p(y \mid \mathbf{x})\, d\Pi(p \mid \mathbf{x}, \mathcal{D}_n). \tag{1}$$

At deployment, pairs follow a different distribution Π' with size marginal Ψ'.

In the following, we study PFNs through the lens of risk minimization. Risks average over a task, a context $\mathcal{D}_n \sim p^{\otimes n}$ and an independent query $(\mathbf{X}, Y) \sim p$. We write $\mathbb{E}_p[\cdot] := \mathbb{E}_{\mathcal{D}_n \sim p^{\otimes n}} \mathbb{E}_{(\mathbf{X},Y)\sim p}[\cdot]$ for a fixed pair (p, n) and $\mathbb{E}_\Pi[\cdot] := \mathbb{E}_{(p,N)\sim\Pi} \mathbb{E}_p[\cdot]$ when the pair is drawn as well, likewise for Π'. Random variables are capitalized.

2.2 Risk Under a Task Distribution

A PFN with parameters $\boldsymbol{\theta} \in \Theta$ is a measurable map $q_{\boldsymbol{\theta}} \colon (\mathbf{x}, \mathcal{D}_n) \mapsto q_{\boldsymbol{\theta}}(\cdot \mid \mathbf{x}, \mathcal{D}_n)$ into the distributions on $\mathcal{Y}$. Deployed on Π', its risk under the log loss is

$$R_{\Pi'}(q_{\boldsymbol{\theta}}) := \mathbb{E}_{\Pi'}\big[-\log q_{\boldsymbol{\theta}}(Y \mid \mathbf{X}, \mathcal{D}_N)\big]. \tag{2}$$

The following proposition characterizes the optimal predictor under an arbitrary task distribution.

Proposition 1. *Let $R_{\Pi'}(\pi_{\Pi'})$ be finite. Then for every predictor q,*

$$R_{\Pi'}(q) - R_{\Pi'}(\pi_{\Pi'}) = \mathbb{E}_{\Pi'}\big[D_{\mathrm{KL}}\big(\pi_{\Pi'}(\cdot \mid \mathbf{X}, \mathcal{D}_N) \,\big\|\, q(\cdot \mid \mathbf{X}, \mathcal{D}_N)\big)\big] \geq 0, \tag{3}$$

with equality if and only if $q = \pi_{\Pi'}$ almost surely under Π'. In particular, $\pi_{\Pi'}$ is the unique minimizer of $R_{\Pi'}$ and $R_{\Pi'}(\pi_{\Pi'})$, the Bayes risk *of Π', is the lowest risk any predictor can attain on Π'.*

The proof is given in Appendix B.1. A PFN $q_{\boldsymbol{\theta}}$ is pretrained on the population objective

$$\mathcal{L}(\boldsymbol{\theta}) := R_\Pi(q_{\boldsymbol{\theta}}). \tag{4}$$

With $\Pi' = \Pi$, Proposition 1 thus gives the pretraining objective: *a PFN is trained to minimize its risk under its own prior*, the minimizer over all predictors is π_Π (Müller et al., 2021), and the excess risk

$$\mathcal{A}_\Pi(q_{\boldsymbol{\theta}}) := R_\Pi(q_{\boldsymbol{\theta}}) - R_\Pi(\pi_\Pi) = \mathbb{E}_\Pi\big[D_{\mathrm{KL}}\big(\pi_\Pi \,\|\, q_{\boldsymbol{\theta}}\big)\big] \geq 0 \tag{5}$$

is the *amortization error*. This error is the distance, in expected KL, of the network from the PPD it is supposed to amortize.

In practice, Eq. (4) is approximated by empirical risk minimization over M datasets $\mathcal{D}^{(j)}$ whose task and size (p_j, n_j) are drawn from the prior,

$$\mathcal{L}_{\mathrm{emp}}(\boldsymbol{\theta}) := -\tfrac{1}{M} \textstyle\sum_{j=1}^M \log q_{\boldsymbol{\theta}}\big(y_j \mid \mathbf{x}_j, \mathcal{D}^{(j)}\big), \tag{6}$$

optimized with first-order stochastic gradient methods such that, in general, we have no guarantee how close we get to the target π_Π.

Figure 1 places the amortization error among the two other sources of error of a deployed PFN: the *context error*, incurred even by π_Π because a finite context leaves the task uncertain, and the *prior error*, incurred because real tasks follow $\Pi' \neq \Pi$. We focus on the former and work towards its precise relation to the other two in Proposition 2. As a first step, we propose how to analyse the amortization error for a tractable but realistic reference prior in the following section.

2.3 Measuring Amortization Error

Measuring $\mathcal{A}_\Pi(q) = \mathbb{E}_\Pi[D_{\mathrm{KL}}(\pi_\Pi \| q)]$ directly requires knowledge of the true PPD, which is intractable for the priors used by current tabular foundation models. The evidence that trained PFNs closely track the PPD they amortize comes from Müller et al. (2021, 2024). The considered settings are low-dimensional compared to current structural causal model (SCM) prior (Qu et al., 2026; Hollmann et al., 2025), none use a current PFN architecture at scale, and only small context sizes

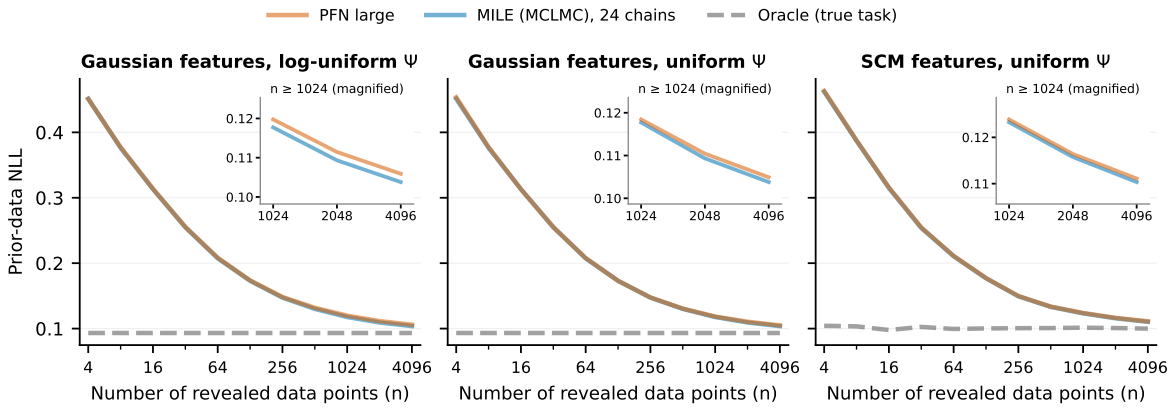


Figure 2: Comparison of PPD approximation quality to MCMC reference on a classification task across both Gaussian and TabICL prior feature distributions (SCM prior). The labeling mechanism is a Bayesian neural network with two hidden layers with 16 neurons each, and the large TabICL variant is trained on these priors from scratch. The PFN approximates the ground-truth PPD provided by the MCMC reference very well across different context sizes (Figure inspired by Müller et al., 2021). Each point is the risk on datasets of context size n; Ψ is the size distribution of pretraining.

are considered. Furthermore, the MCMC reference performed worse than the PFN under consideration.

To fill this gap, we measure the amortization error not only in the setting introduced in Müller et al. (2021) with standard Gaussian features but also in a considerably more realistic setting by keeping the covariate generator of a state-of-the-art synthetic prior, so that the context tables have the marginal structure of current SCM priors (mixed feature types, correlations, non-Gaussian marginals). We replace the random mechanism mapping covariates to targets by a Bayesian neural network (BNN) $f_{\boldsymbol{w}}$ defining the likelihood $y \mid \mathbf{x}, \boldsymbol{w} \sim p(y \mid f_{\boldsymbol{w}}(\mathbf{x}))$ with $\boldsymbol{w}$ drawn from a Gaussian prior, analogous to the BNN prior introduced in Hollmann et al. (2023). Because the covariate generator and $\boldsymbol{w}$ are drawn independently, this prior factorizes. Conditioning on $\mathbf{x}^*$ leaves the posterior over $\boldsymbol{w}$ unchanged, and the PPD reduces to

$$\pi_\Pi(y \mid \mathbf{x}^*, \mathcal{D}_n) = \int p\big(y \mid f_{\boldsymbol{w}}(\mathbf{x}^*)\big)\, d\Pi(\boldsymbol{w} \mid \mathcal{D}_n). \quad (7)$$

Making use of recent advances in both MCMC algorithm development (Robnik and Seljak, 2024) as well as sampling-based inference for BNNs (Sommer et al., 2025, 2026), we compare the prediction of PFNs against the PPD they are trained to amortize to obtain an empirical lower bound on the approximation error. MCMC yields a reference $\hat{\pi}_\Pi$ against which $D_{\mathrm{KL}}(\hat{\pi}_\Pi \| q)$ can be computed for each pair $(\mathbf{x}^*, \mathcal{D}_n)$. In Fig. 2, we plot the aggregate NLL of a PFN against a large-budget MCMC reference. In the inset plots, it is visible that the MCMC reference, where 24 MCMC chains are sampled per task, still dominates the PFN, however, the amortization gap appears small and mostly dependent on what datasets are probable under the prior (deviations at small context settings for Ψ uniform vs. deviations at large context settings for Ψ loguniform). Details are given in Appendix C.

While this approach is sufficient to study the general capabilities of transformers to approximate the PPD and measure quantitatively how good the approximation is in terms of the amortization error, two aspects complicate the analysis once we move to production-grade synthetic priors and deployment on real-world datasets: We cannot compute the reference PPD, and deployment under a distribution Π' different from the training distribution Π introduces a distribution shift (see Appendix C). The following studies these problems in more detail.

3 AMORTIZATION ERROR UNDER DEPLOYMENT

3.1 Amortization Error Without a Reference

Beyond tractable priors, we cannot compute a reference PPD, and the amortization error has to be studied without an (approximate) ground-truth. Viewing the PFN with parameters $\hat{\boldsymbol{\theta}}$ via empirical risk minimization, the amortization error splits into three components

$$\mathcal{A}_\Pi(q_{\hat{\boldsymbol{\theta}}}) = \underbrace{R_\Pi(q_{\hat{\boldsymbol{\theta}}}) - R_\Pi(q_{\tilde{\boldsymbol{\theta}}_M})}_{\text{optimization}} + \underbrace{R_\Pi(q_{\tilde{\boldsymbol{\theta}}_M}) - R_\Pi(q_{\boldsymbol{\theta}^*})}_{\text{estimation}} + \underbrace{R_\Pi(q_{\boldsymbol{\theta}^*}) - R_\Pi(\pi_\Pi)}_{\text{approximation}}, \quad (8)$$

where $\tilde{\boldsymbol{\theta}}_M$ minimizes the empirical risk for M training samples and $\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\boldsymbol{\theta})$, i.e., minimizes the population risk (4). The **estimation error** is the part

that classical learning theory ties to the number of pretraining datasets M and expects to decrease at a $1/M$ rate (Nagler, 2023). This is where PFNs profit from effectively unlimited synthetic data. The **approximation error** is a property of the hypothesis class and is reduced only by enlarging it, which extensive work on architectures attempts (Qu et al., 2026; Grinsztajn et al., 2025; Kong et al., 2026). The **optimization error** is a consequence of the optimizer and for overparameterized networks trained on a non-convex risk, it is not guaranteed to decrease with model size or training duration.

To confirm that the estimation error vanishes as expected, we can study K models that share all optimizer settings but differ in seed and training data, at several scales $M_1 < \cdots < M_L$, and compute

$$\widehat{EE}(M) = \tfrac{1}{K}\sum_{k=1}^{K} \mathcal{L}_{\text{test}}(\boldsymbol{\theta}_M^{(k)}) - \mathcal{L}_{\text{test}}(\boldsymbol{\theta}_{\text{ref}}), \quad (9)$$

where $\boldsymbol{\theta}_{\text{ref}}$ is a reference checkpoint with $M_{\text{ref}} \gg M_L$. Averaging over the K runs cancels the zero-mean optimization noise of individual runs.

3.2 Shared & Removable Amortization Error

Optimization and approximation error cannot be isolated in the same way, since neither $\tilde{\boldsymbol{\theta}}_M$ nor $\boldsymbol{\theta}^*$ nor π_Π is tractable. What is observable is the functional spread across K independently trained ensemble members $q_1, \ldots, q_K$ with identical M, model size, and optimizer settings. We measure this spread by the generalized Jensen–Shannon divergence $\widehat{AU}_{\text{JSD}}$ between the members' predictive distributions and by the pairwise argmax disagreement rate $\widehat{AU}_{\text{dis}}$, both averaged over test points (definitions in Appendix D). Neither requires labels. The following classical identity (Cover and Thomas, 2006) connects this spread to predictive performance.

Lemma 1. *For any distribution p on $\mathcal{Y}$ and predictors $q_1, \ldots, q_K$ with mixture $\bar{q} = \frac{1}{K}\sum_{k=1}^{K} q_k$,*

$$\tfrac{1}{K}\sum_{k=1}^{K} D_{\text{KL}}(p \,\|\, q_k) = D_{\text{KL}}(p \,\|\, \bar{q}) + \Delta_p, \quad (10)$$

where $\Delta_p := \mathbb{E}_{y\sim p}\big[\log \bar{q}(y) - \frac{1}{K}\sum_{k=1}^{K} \log q_k(y)\big] \geq 0$, with equality if and only if all q_k agree p-almost everywhere.

Δ_p is the *averaging gain*, i.e., the amount by which the mixture improves on its average member in expected log loss under p. Applied with $p = \pi_\Pi$, Lemma 1 splits the amortization error itself:

Corollary 1. *For PFNs $q_1, \ldots, q_K$ with mixture $\bar{q}$,*

$$\tfrac{1}{K}\sum_k \mathcal{A}_\Pi(q_k) = \underbrace{\mathcal{A}_\Pi(\bar{q})}_{\text{shared}} + \underbrace{\mathbb{E}_\Pi\big[\Delta_{\pi_\Pi}\big]}_{\text{removable } \geq 0}. \quad (11)$$

Thus, the average amortization error of K PFNs splits into two parts that differ in what can be observed without the PPD. The first, $\mathcal{A}_\Pi(\bar{q})$, is the error the members have in common as they jointly deviate from π_Π, which cannot be measured without a reference as in Section 2.3. The second term is the part of the amortization error that differs between the members and does not require a reference to compute. It equals the log-loss gain $\frac{1}{K}\sum_k R_\Pi(q_k) - R_\Pi(\bar{q})$ of the mixture over its average member, which can be estimated on datasets drawn from the prior, and it vanishes if and only if the members agree. The disagreement metrics $\widehat{AU}_{\text{JSD}}$ and $\widehat{AU}_{\text{dis}}$ are therefore label-free probes of this visible part, and blind to the shared one.

Revisiting Eq. (8), if the estimation error vanishes at the expected rate while disagreement persists, the remaining amortization error is consistent with optimization error, possibly combined with residual approximation error: independent runs reach functionally distinct solutions that more data does not reconcile. To test whether the removable part is driven to zero by the usual scaling levers, we study $\widehat{EE}$ and both disagreement metrics along two axes, the number of pretraining datasets M (data axis) and the model size (capacity axis). Results are depicted in Fig. 3.

Under $\Pi' \neq \Pi$, Proposition 1 says that the optimal predictor is $\pi_{\Pi'}$, not the π_Π the PFN was trained towards. Deploying a PFN on real data is thus a *distribution shift* from Π to Π'. The next result makes its cost explicit and reconciles in what sense model averaging can be beneficial.

3.3 Decomposing the Deployment Error

If the PFN training on Π does not perfectly align with the deployment distribution Π', this difference needs to be taken into account. To locate the deployment pairs (p, n) relative to the prior, the Lebesgue decomposition splits the deployment distribution uniquely into a part the prior can express $\tilde{\Pi}$, and a part $\Pi'_\perp$ it cannot,

$$\Pi' = \lambda\,\tilde{\Pi} + (1-\lambda)\,\Pi'_\perp, \qquad \tilde{\Pi} \ll \Pi, \quad \Pi'_\perp \perp \Pi, \quad (12)$$

with density $\tilde{r}_\Pi = d\tilde{\Pi}/d\Pi$ on the absolutely continuous part. The weight $\lambda \in [0, 1]$ is the fraction of deployment pairs reachable by reweighting the prior. The singular part $\Pi'_\perp$ lives on pairs to which Π assigns probability zero, and no reweighting reaches it. These include tasks the prior cannot generate as well as context sizes outside the pretraining range. As a baseline for the deployment risk, let $H_{\Pi'} := \mathbb{E}_{\Pi'}[-\log p(Y \mid \mathbf{X})]$ be the expected conditional entropy of the deployment tasks, i.e., the risk of an oracle that knows each task p and predicts with $p(y \mid \mathbf{x})$. The excess risk of a PFN over this oracle then splits as follows.